



Interfaces graphiques

UI en Java

Concepteur Développeur d'Applications



COMPARAISON

Il existe deux librairies principales pour réaliser des interfaces graphiques en Java

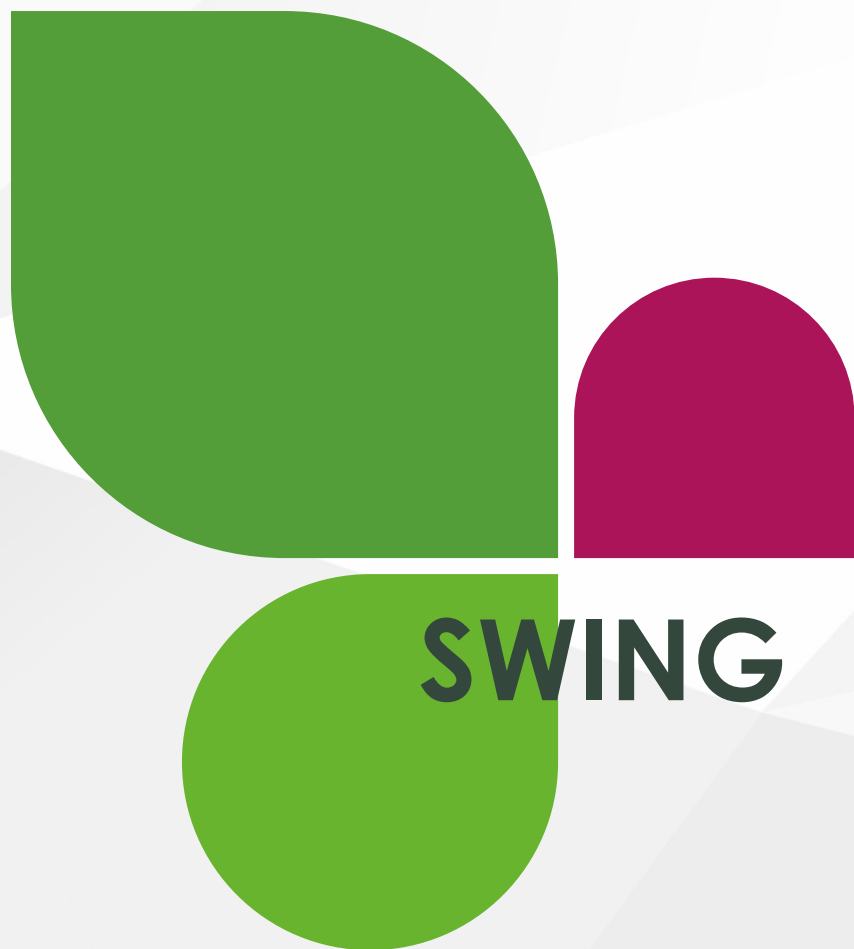
Mais vous pouvez trouver des Framework comme Vaadin par exemple

SWING

- L'approche Swing : une application doit fonctionner et se présenter à l'identique, qu'elle que soit la plateforme considérée. Ce n'est plus l'OS qui va pixeliser les différents éléments graphiques de vos IHM, mais bien l'API Java Swing.
- Du coup, si un composant graphique n'est pas proposé par un système d'exploitation, il n'y a pas de soucis : Swing vous le proposera quand même.
- Donc un point très positif à utiliser Swing réside dans la richesse des composants proposés (en nombre, largement supérieur à ceux proposés par la librairie AWT).

JAVAFX

- JavaFX est désormais l'API d'interface graphique principale du Java SE.
- Dans sa conception, elle est plus moderne que Swing et permet de produire des interfaces graphiques pouvant facilement être utilisées sur différents types d'écrans (écran standard de PC, smartphone & tablettes et applications Web).
- JavaFX ressemble, à plusieurs égards, à certaines API des développement Web dans le sens où les vues peuvent se définir via un formalisme XML (FXML), le code Java ne contenant quasiment plus que les gestionnaires d'événements.



Concepteur Développeur d'Applications

Introduction

Swing est une bibliothèque graphique pour Java, offrant la possibilité de créer des interfaces graphiques.

Il utilise le principe [Modèle-Vue-Contrôleur](#) où les composants Swing jouent le rôle de la Vue.

- *Avant lui, l'utilisation AWT basé sur les composants système, rapide mais pas exportable et en manque de composants.*

Aujourd'hui, Swing n'est plus "vraiment tendance" et a été remplacé par JavaFX depuis la version 8 de Java, mais reste pratique pour l'apprentissage.

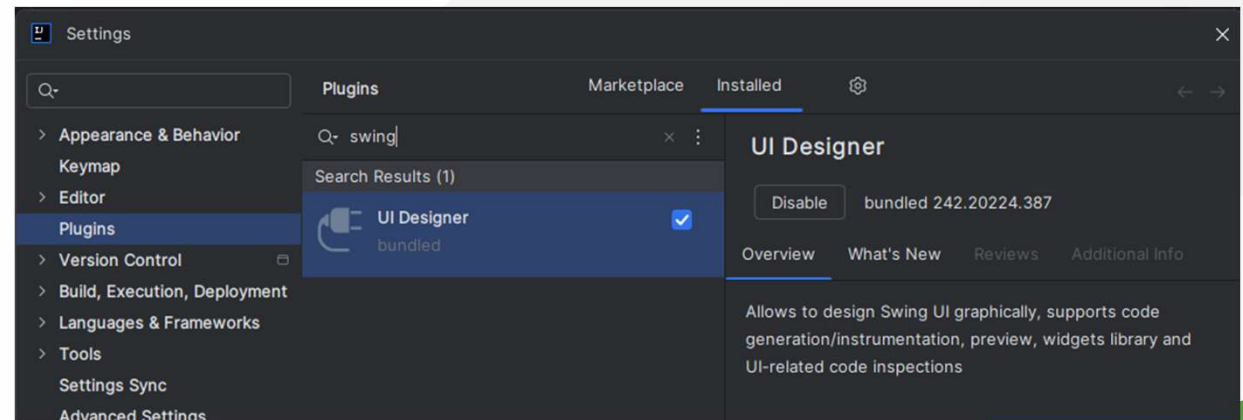


Installation

La bibliothèque **Swing** est disponible dans la JDK sauf qu'aujourd'hui elle n'est plus maintenue sauf pour la correction de bug.

- Afin de pouvoir travailler de manière optimale, nous allons également installer un composant à notre IDE afin de pouvoir créer des composants graphiques facilement.

Après installation, vous aurez la possibilité de créer une application intégrant directement le premier composant Swing : la fenêtre principale de l'application.



Les Layout

Les différents types de couches

Layout Manager

Problème

- Une difficulté classiquement rencontrée lors de la mise en œuvre d'interfaces graphiques
 - positionner les différents composants graphiques dans une zone afin d'obtenir une sensation d'équilibre dans l'IHM (l'interface graphique).
- 1. Si au contraire les éléments graphiques sont trop ramassés dans un coin de l'IHM, l'esthétisme de l'interface risque (peut-être) de rebuter les utilisateurs.
- 2. De même, il faudrait, lorsque l'on retaille la fenêtre, que les composants qu'elle contient soient repositionnés et retaillés harmonieusement.

Solution

- Pour répondre à ces besoins, et pour automatiser les calculs de positionnement et de "retailage", les librairies AWT et Swing utilisent **le concept commun de "layout manager" (gestionnaires de positionnement / stratégies de positionnement)**.
 - Il existe plusieurs classes de **layout managers**, chacune d'elle possédant sa propre logique de positionnement des éléments (positionnement en lignes, en grille, ...).

Une stratégie de placement à un conteneur

Comportement

- La première chose à savoir, est qu'une stratégie de placement se définit au niveau d'un conteneur `java.awt.Container`.
 - Toutes les classes Swing dérivent (directement ou indirectement de `java.awt.Container`).
- Chaque type de conteneur utilise une stratégie par défaut :
 - par exemple une fenêtre Swing possède un conteneur (accessible via la méthode `getContentPane`) qui par défaut utilise une stratégie de type `java.awt.BorderLayout`

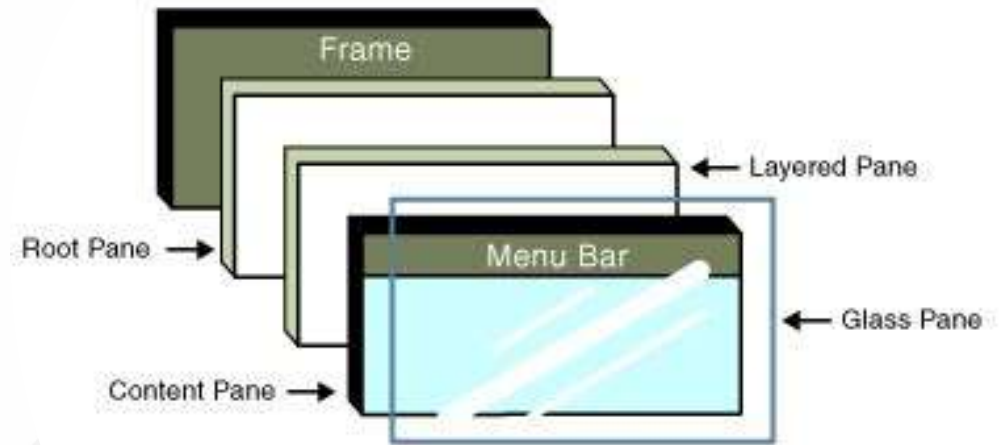
Affectation

- Bien entendu, une stratégie de placement se définit en créant une instance de l'une des classes de stratégie de placement.
- Une fois cet objet instancié, il ne reste plus qu'à l'affecter à un conteneur.
- Pour ce faire, on utilise la méthode `setLayout`.
 - Elle prend un unique paramètre : le `LayoutManager`.
 - *Si vous passez la valeur **null** en paramètre, aucune stratégie de placement ne sera utilisée : il faut alors positionner au en absolu vos composants.*

Composition

Les conteneurs : Une **JFrame** est découpée en plusieurs parties :

- la fenêtre.
- le **RootPane**
 - container principal qui contient les autres
- le **LayeredPane**
 - forme juste un panneau composé du ContentPane et de la barre de menu (MenuBar)
- le **GlassPane**
 - utilisé pour intercepter les actions de l'utilisateur
- le **ContentPane**
 - le composant qui contient la partie utile de la fenêtre, c'est-à-dire, celle dans laquelle on va afficher nos composants.



From Sun Microsystems(tm) Website

les conteneurs

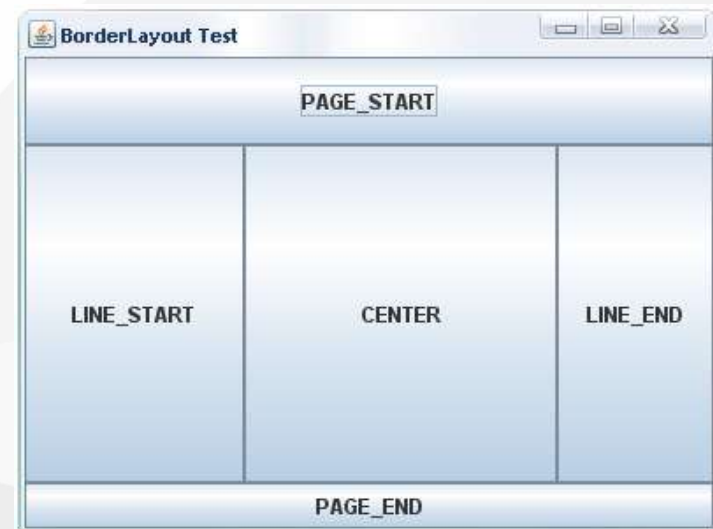
En Swing, on peut dire qu'il y a deux types de composants : les containers et les composants.

1. Les containers sont destinés à contenir d'autres containers et des composants
 2. Les composants sont destinés à quelque chose de précis, par exemple afficher du texte ou permettre à l'utilisateur de saisir du texte.
- Le `ContentPane` est donc un container. Pour ajouter des composants dans la fenêtre :
 1. on va donc créer notre propre `JPanel` en lui indiquant un `layout`,
 2. ajouter des composants à l'intérieur du `JPanel`
 3. indiquer à la fenêtre que notre `JPanel` sera le `ContentPane`.



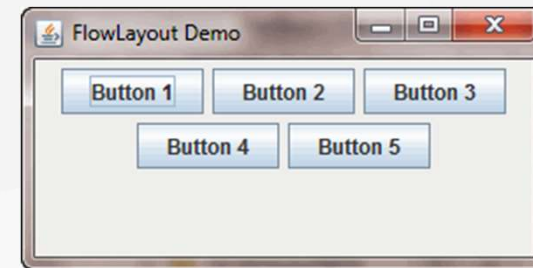
La stratégie BorderLayout

- Cette stratégie permet d'opérer une division de l'espace utilisable au sein d'un conteneur en cinq zones.
- Quatre de ces cinq zones sont placés de chaque côté du conteneur
 - BorderLayout.NORTH
 - BorderLayout.WEST
 - BorderLayout.EAST
 - BorderLayout.SOUTH
- La cinquième zone est placée au centre du conteneur et est entourée par les quatre autres.
 - BorderLayout.CENTER



La stratégie FlowLayout

- Si vous l'affectez à un conteneur, celui-ci va tenter de mettre le plus de composants possibles sur une ligne.
- Dès que la ligne est pleine (en fonction de la taille du conteneur), le layout positionnera les prochains éléments sur une nouvelle ligne, et ainsi de suite.
- Vous pouvez de plus configurer votre layout en spécifiant quel type d'alignement vous souhaitez utiliser.
- En utilisant la méthode `setAlignment` : des attributs (constants) de la classe `FlowLayout` sont prédéfinis
 - `FlowLayout.LEFT`,
 - `FlowLayout.CENTER`,
 - `FlowLayout.RIGHT`



La stratégie GridLayout

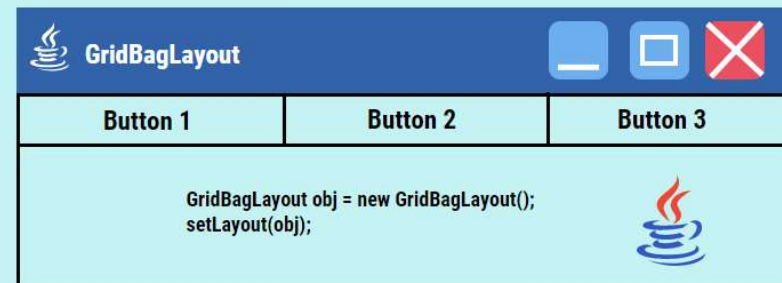
- Celle-ci permet de disposer vos composants dans une grille constituée de lignes et de colonnes.
- Pour ce faire, le constructeur de la classe GridLayout peut accepter deux paramètres :
 - le nombre de lignes
 - le nombre de colonnes.



La stratégie GridBagLayout

- En ce qui concerne le `GridBagLayout`, le principe est exactement le même.
- Au lieu d'être limité à cinq zones, le `GridBagLayout` agence les différents composants sur une grille virtuelle dont chaque ligne peut avoir une hauteur différente des autres et chaque colonne une largeur différente des autres.
- Chaque composant peut s'étendre sur plusieurs cellules aussi bien horizontalement que verticalement dans cette grille.
- Le `GridBagLayout` est donc tellement flexible que de simples constantes comme dans le cas du `BorderLayout` ne suffisent pas.
- Nous devons alors spécifier cette contrainte au travers d'un objet de la classe `GridBagConstraints`.

GridBagLayout in Java



www.educba.com

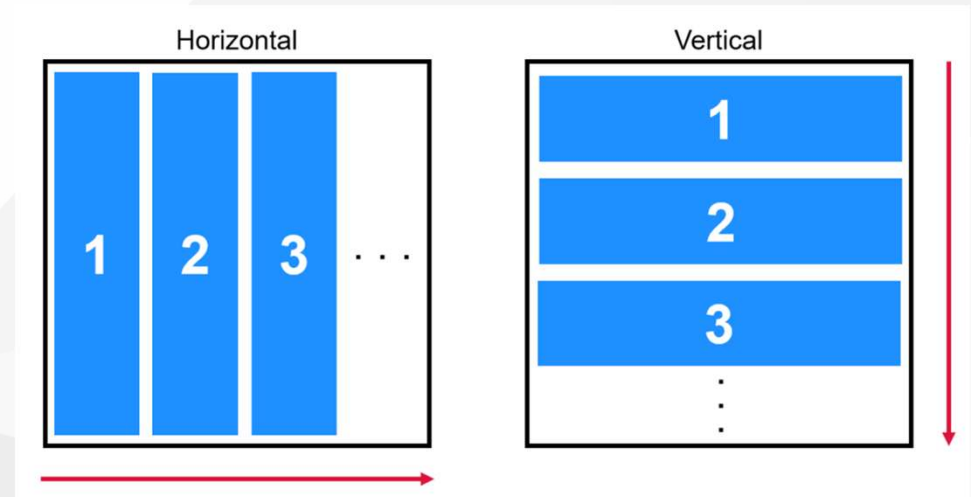
La stratégie CardLayout

- Ce gestionnaire de répartition permet d'avoir plusieurs composants graphiques tels que l'un d'entre eux seulement soit visible à un moment donné.
- Tout se passe comme si on avait un jeu de cartes et qu'on pouvait choisir la carte visible.



La stratégie BoxLayout

- Le gestionnaire de répartition **BoxLayout** permet de mettre des composants en ligne ou en colonne.
- Contrairement au **GridLayout**, les sous-composants ne sont pas obligés d'avoir toute la même largeur et la même hauteur.
- Ce gestionnaire de répartition offre beaucoup de fonctionnalités.



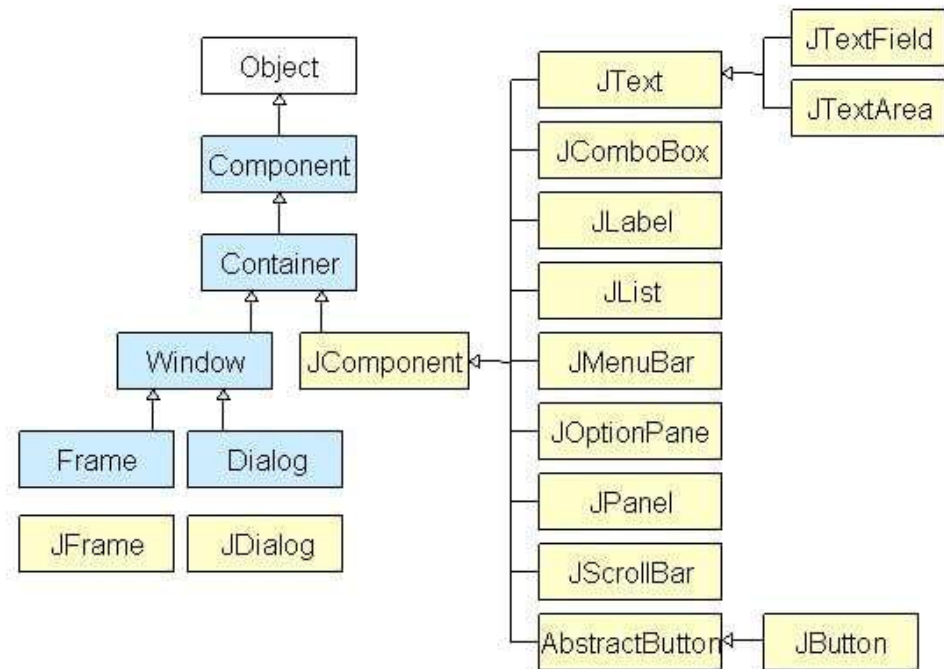
Les composants graphiques

Construire nos UI



Les composants

Voici les différents composants qui sont proposés par SWING



Les fenêtres

Il existe plusieurs types de fenêtres dans Swing

- **JWindow** : les fenêtres basiques.
 - C'est juste un conteneur que vous pouvez afficher sur votre écran. Il n'a pas de barre de titre, pas de boutons de fermeture/redimensionnement et n'est pas redimensionnable par défaut. (utilisé pour faire patienter)
- **JDialog** : les boîtes de dialogue.
 - Ce type de fenêtre peut être modal, c'est-à-dire qu'elle bloque une autre fenêtre tant qu'elle est ouverte. Elles sont destinées à travailler de pair avec la fenêtre principale que nous allons voir maintenant.
- **JFrame** : les fenêtres de votre application.
 - Elle n'est dépendante d'aucune autre fenêtre et ne peut pas être modale. Elle a une barre de titre et peut accueillir une barre de menu. Elle possède un bouton de fermeture, un bouton de redimensionnement et un bouton pour l'iconifier.

Les fenêtres

JDialog

- Permet de créer des popup pouvant servir à afficher des informations, des questions etc...
- Généralement associé avec JOptionPane qui permet de créer des popup très facilement

```
// PopUp
// instantiation
JDialog dialog = new JDialog();
//On lui donne une taille
dialog.setSize(300, 200);
//On lui donne un titre
dialog.setTitle("Première fenêtre");
//On la rend visible
dialog.setVisible(true);
```

JFrame

- Permet de créer des fenêtres.
- Une fenêtre = une classe
- [Héritant de JFrame](#)
- Un constructeur qui construit la fenêtre en la paramétrant :
 - setTitle()
 - setSize()
 - setLocationRelativeTo()
 - setResizable()
 - setDefaultCloseOperation()
 - setLayout() : IMPORTANT

Afficher du texte JLabel

Il y a deux sortes de composants textes dans une interface graphique

1. Les composants de saisie :
 - Ce sont les composants qui permettent à l'utilisateur de saisir du texte. On va les décrire plus loin.
2. Les composants d'affichage **JLabel** :
 - Ce sont les composants qui permettent d'afficher du texte. Ce sont sur ceux-là qu'on va se concentrer. Je dis ceux-là, mais en fait, on ne va en utiliser qu'un.

Illustration :

1. Création d'un label
2. Modification de son alignement
3. Placement du label dans le Panel central

```
// création d'un label
JLabel label = new JLabel("Bienvenue dans ma modeste application");
// Positionnement du label
label.setHorizontalAlignment(SwingConstants.CENTER);
// Ajout du label dans le panel de la Frame
frame.getContentPane().add(label, BorderLayout.NORTH);
```

Demander du texte JTextField et cie

Pour cela, il existe plusieurs composants :

Le **JTextField** : C'est le plus simple des composants textes. Il permet d'entrer un texte sur une seule ligne.

Le **JTextArea** : Il permet d'entrer un texte complet sur plusieurs lignes.

Le **JEditorPane** : Très complet, vous pouvez modifier la police, la taille, la couleur de votre texte. Il permet même d'afficher des pages html. Vous pouvez y insérer du texte et des images.

Le **JTextPane** : C'est le composant texte le plus évolué de Swing. Il permet la même chose que le **JEditorPane** mais il permet également d'insérer des composants personnalisés, c'est-à-dire que vous pouvez y insérer des composants Swing.

Illustration :

- Pour bien gérer le champ de texte, je le positionne dans un Panel
- Création de mon **JPanel**
- Ajout du panel dans le **ContentPane**
- Création du **JTextField**
- Ajout d'options et de propriétés
- Ajout dans le panel dédié

```
// Création d'un champs de saisie
// creation d'un panel qui va contenir nom textfield
JPanel panel = new JPanel();
// ajout de panel dans le contentPane
frame.getContentPane().add(panel, BorderLayout.CENTER);
// instantiation du textfield
JTextField textField = new JTextField();
// aide à la saisie
textField.setToolTipText("taper votre réponse");
//On lui donne un nombre de colonnes à afficher
textField.setColumns(10);
// ajout du textfield dans le panel
panel.add(textField);
```

Manipulation des zones de texte

Pour exploiter nos JLabel et nos JTextField, on a accès à tout un ensemble de méthodes.

Par exemple, pour récupérer la saisie utilisateur et l'afficher dans un label par exemple, il suffit de simplement utiliser un **getter** et un **setter** provenant des classes JLabel et JTextField

```
private void valider(ActionEvent e) {  
    System.out.println(e.getActionCommand().toString());  
    System.out.println(e.getSource().toString());  
  
    this.label.setText(this.getTextField().getText());  
}
```

Ajout de boutons JButton

La principale fonctionnalité d'une application graphique est la programmation événementielle, c'est-à-dire le fait que l'utilisateur puisse déclencher des événements et réagir à ce qui se passe dans la fenêtre.

Au contraire d'un programme en mode console, dans lequel le programme régit les actions de l'utilisateur à sa guise.

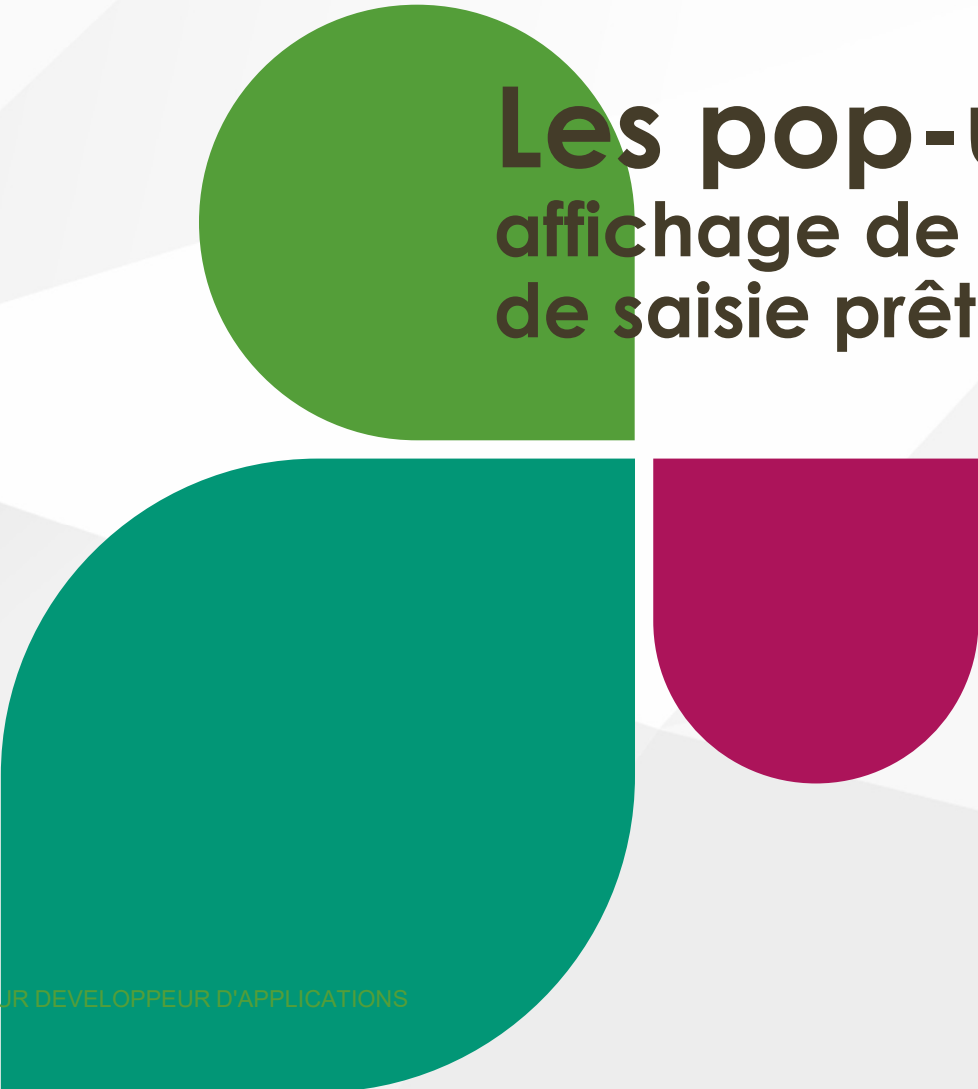
Pour que l'utilisateur puisse interagir avec le programme, on dispose par exemple, des boutons.

JButton implique une action soit une gestion d'événements.

Illustration :

- Pour bien gérer les boutons, je le positionne dans un Panel
 - Création de mon JPanel
 - Ajout du panel dans le ContentPane
 - Création des boutons
 - Ajout dans le panel dédié

```
// creation d'un panel qui va contenir mes boutons
JPanel panelButton = new JPanel();
frame.getContentPane().add(panelButton, BorderLayout.SOUTH);
// Creation de mes deux boutons
JButton valid = new JButton("Valider");
JButton cancel = new JButton("Annuler");
panelButton.add(valid);
panelButton.add(cancel);
```

Les pop-ups

affichage de messages, affiche
de saisie prête à l'emploi

Les fenêtres popUp

Il existe un composant très intéressant, le **JOptionPane** qui possède plusieurs méthodes **statiques** permettant d'ouvrir d'afficher diverses boîtes de dialogue :

- Les boîtes de messages permettent d'afficher un message pour l'utilisateur.
- Les boîtes de saisie permettent de demander une chaîne de caractères à l'utilisateur
- Les boîtes de confirmation demandent une confirmation à l'utilisateur.
- Les boîtes de choix permettent de donner le choix entre plusieurs choses, une liste déroulante en quelque sorte

Constructeurs de JOptionPane	Description
JOptionPane()	utilisé pour créer un JOptionPane avec un message de test.
JOptionPane(Object message)	utilisé pour créer une instance de JOptionPane pour afficher un message.
JOptionPane(Object message, int messageType)	utilisé pour créer une instance de JOptionPane pour afficher un message avec le type de message spécifié et les options par défaut.

Les fenêtres popUp

Les méthodes couramment utilisées.

- **createDialog(String title)**
 - utilisé pour créer et renvoyer un nouveau JDialog sans parent avec le titre spécifié.
- **showMessageDialog(Component parentComponent, Object message)**
 - Il est utilisé pour créer une boîte de message d'information intitulée « Message ».
- **showMessageDialog(Component parentComponent, Object message, String title, int messageType)**
 - Il est utilisé pour créer une boîte de message avec un titre et un type de message donnés.
- **showConfirmDialog(Component parentComponent, Object message)**
 - Il est utilisé pour créer une boîte de dialogue avec les options Oui, Non et Annuler; avec le titre, Sélectionnez une option.
- **showInputDialog(Component parentComponent, Object message)**
 - Il est utilisé pour afficher une boîte de question-message demandant l'entrée de l'utilisateur.
- **void setInputValue(Object newValue)**
 - Il est utilisé pour définir la valeur entrée par l'utilisateur.

Int MessageType

Le paramètre **messageType** utilisé dans les méthodes correspond au type de message qu'on souhaite afficher.

Il suffit de faire appel aux constantes fournies par la classe.

Fields	Description
int ERROR_MESSAGE	Message d'erreur
int INFORMATION_MESSAGE	Message d'information
int WARNING_MESSAGE	Message de warning
int YES_NO_OPTION	Message dialogue Oui / Non
int YES_NO_CANCEL_OPTION	Message dialogue Oui / Non / Annuler
int OK_CANCEL_OPTION	Message dialogue Oui / Annuler

Illustration de messages pop-up

Message informatif

```
private void quitter(ActionEvent e) {  
    System.out.println(e.getActionCommand().toString());  
    System.out.println(e.getSource().toString());  
  
    int sortie = JOptionPane.showConfirmDialog(frame, "Etes-vous sûr ?");  
  
    if (sortie == JOptionPane.YES_OPTION) {  
        System.exit(0);  
    }  
}
```

Message de confirmation

```
private void quitter(ActionEvent e) {  
    System.out.println(e.getActionCommand().toString());  
    System.out.println(e.getSource().toString());  
  
    int sortie = JOptionPane.showConfirmDialog(frame, "Etes-vous sûr ?",  
        "Quitter", JOptionPane.YES_NO_OPTION);  
  
    if (sortie == JOptionPane.YES_OPTION) {  
        System.exit(0);  
    }  
}
```

Les composants

JComboBox et JTable

Liste de choix JComboBox

Il existe 2 manières de manipuler des JComboBox :

1. soit on utilise directement les méthodes de manipulations des éléments de la JComboBox

- La première manière de faire, c'est simplement d'utiliser les méthodes que nous propose JComboBox :
 - **addItem(Objet item)**
 - ajouter un nouvel objet à la liste
 - **removeItem(Objet item)**
 - enlever l'objet de la liste
 - **removeAllItems()**
 - vider la liste déroulante
 - **getSelectedItem()**
 - retourne l'objet qui est actuellement sélectionné

```
Object[] elements = new Object[]{"Element 1", "Element 2", "Element 3", "Element 4", "Element 5"};

liste1 = new JComboBox<Object>(elements);
panelText.add(liste1);
textField.setText(liste1.getSelectedItem().toString());
```

Liste de choix JComboBox

2. soit on développe notre propre modèle de liste
 - Dans cet exemple, je fais le choix de définir un modèle StringModel qui contiendra les éléments à afficher dans notre ComboBox.

1. Cette fois-ci, on ne va pas manipuler directement le JComboBox, mais son modèle soit **StringModel**.
2. Pour créer un modèle de liste, il suffit d'hériter de **DefaultComboBoxModel**. On peut aussi seulement implémenter **ComboBoxModel**, mais après, il faudra recoder une bonne partie qui est là-même à chaque fois. (voir page suivante).
3. Enfin, appliquer notre modèle à notre **combobox**

```
String[] strings = new String[]{"etat 1", "etat 2", "etat 3", "etat 4", "etat 5"};  
  
liste2 = new JComboBox<Object>(new StringModel(strings) );  
paneltext.add(liste2);
```


La classe Modèle

Ensuite, il faut définir notre classe Modèle.

Comme indiqué précédemment, cette classe hérite de la classe **DefaultComboBoxModel**

Du fait de l'héritage, nous devons redéfinir les méthodes afin de pouvoir manipuler notre modèle.

```
public class StringModel extends DefaultComboBoxModel {
    /**
     *
     */
    private static final long serialVersionUID = 1L;

    private ArrayList<String> strings;

    public StringModel(){
        super();

        strings = new ArrayList<String>();
    }

    public StringModel(String[] strings){
        super();

        this.strings = new ArrayList<String>();
        for(String string : strings){
            this.strings.add(string);
        }
    }

    protected ArrayList<String> getStrings(){
        return strings;
    }

    public String getSelectedString(){
        return (String)getSelectedItem();
    }

    @Override
    public Object getElementAt(int index) {
        return strings.get(index);
    }

    @Override
    public int getSize() {
        return strings.size();
    }

    @Override
    public int getIndexOf(Object element) {
        return strings.indexOf(element);
    }
}
```

Les tableaux JTable

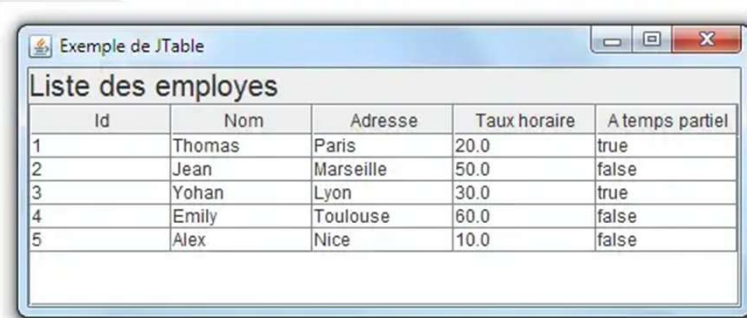
Le composant **JTable** fourni dans l'API Swing de Java est utilisé pour afficher / modifier des données bidimensionnelles. Ceci est équivalent à une feuille de calcul.

- **JTable** fournit des fonctionnalités riches pour gérer l'apparence et le comportement du composant de JTable.
- **JTable** hérite directement de **JComponent** et implémente plusieurs interfaces **listener** telles que **TableModelListener**, **TableColumnModelListener**, **ListSelectionListener**.

- Chaque **JTable** a trois modèles **TableModel**, **TableColumnModel** et **ListSelectionModel**.
 - **TableModel** est utilisé pour spécifier la façon dont les données de la table sont stockées et la récupération de ces données.
 - Les données de JTable sont souvent dans une structure à deux dimensions, comme un tableau à deux dimensions ou un vecteur de vecteurs. TableModel est également utilisé pour spécifier comment les données peuvent être modifiables dans le tableau.
 - **TableColumnModel** est utilisé pour gérer tous les TableColumn en termes de sélection de colonne, l'ordre des colonnes et la taille de la marge.
 - **ListSelectionModel** permet à la table d'avoir différents modes de sélection tels que des sélections par intervalle et intervalle multiple.

Les tableaux JTable

Nous pouvons créer un JTable en utilisant ses constructeurs



Exemple de JTable

Liste des employes

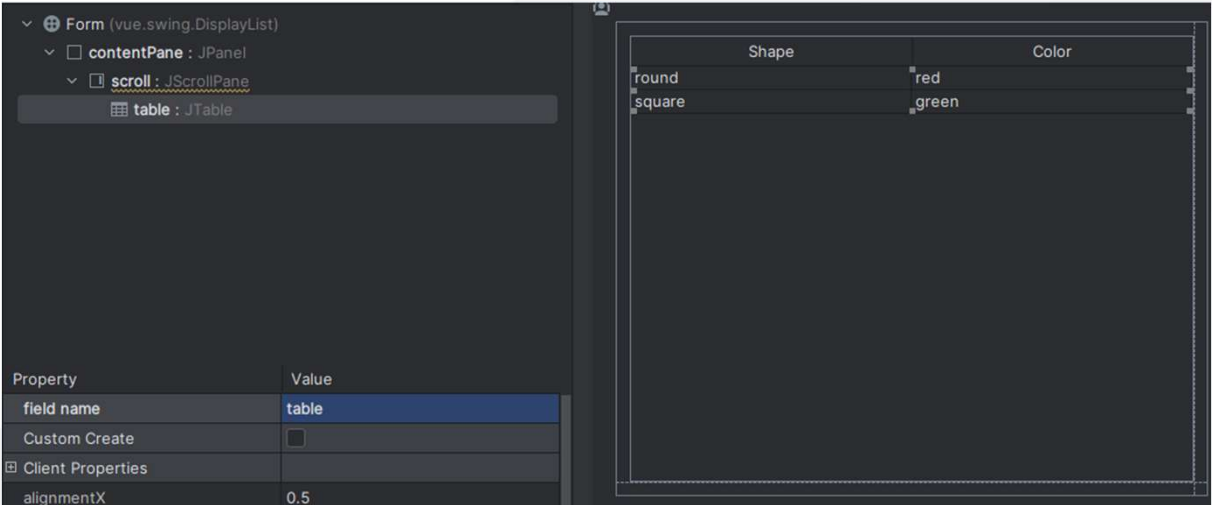
Id	Nom	Adresse	Taux horaire	A temps partiel
1	Thomas	Paris	20.0	true
2	Jean	Marseille	50.0	false
3	Yohan	Lyon	30.0	true
4	Emily	Toulouse	60.0	false
5	Alex	Nice	10.0	false

Constructeurs JTable	Description
JTable()	Créer une table vide
JTable(int rows, int columns)	Créer une table avec des cellules et des lignes vides.
JTable(Object[][] data, Object[] heading)	Créer une table avec des données spécifiées dans un tableau à deux dimensions et un tableau d'en-tête.
JTable(TableModel dm)	Créer une table avec un TableModel donné.
JTable(TableModel dm, TableColumnModel cm)	Créer une table avec un TableModel et TableColumnModel donnés.
JTable(TableModel dm, TableColumnModel cm, ListSelectionModel sm)	Créer une table avec un TableModel, TableColumnModel et ListSelectionModel donné.
JTable(Vector data, Vector heading)	Créer une table avec des données du vecteur et un vecteur d'en-têtes.

Illustration d'une mise en forme de Jtable avec une classe Modèle

Conseils

A partir de GUI **Form**, il est conseillé d'encadrer votre **Jtable** d'un **JScrollPane**



Création de la classe modèle

La classe modèle va vous permettre de créer le modèle des données que la Jtable va afficher.

1. Cette classe hérite de la classe AbstractTableModel, ce qui vous oblige à implémenter les méthodes
2. On doit tout d'abord créer l'entête du tableau. L'entête est un simple tableau contenant les noms des colonnes
3. Et ensuite réécrire les méthodes pour permettre l'utilisation du modèle.

Concepteur Développeur d'Applications

1

2

3

```
public class TableModel<T> extends AbstractTableModel { 2 usages

    private final List<T> data; 3 usages
    private final String[] columnNames; 3 usages
    private final Class<?>[] columnClasses; 2 usages
    private final DateTimeFormatter FORMATTER_DATE_FRENCH = DateTimeFormatter.ofPattern("dd-MM-yyyy"); 1 usage

    public TableModel(List<T> data, String[] columnNames, Class<?>[] columnClasses) { 1 usage
        this.data = data;
        this.columnNames = columnNames;
        this.columnClasses = columnClasses;
    }

    @Override
    public int getRowCount() { return data.size(); }

    @Override
    public int getColumnCount() { return columnNames.length; }

    @Override
    public String getColumnName(int columnIndex) {
        return columnNames[columnIndex];
    }

    @Override
    public Class<?> getColumnClass(int columnIndex) { return columnClasses[columnIndex]; }

    @Override
    public Object getValueAt(int rowIndex, int columnIndex) {
        T item = data.get(rowIndex);
        // Add custom logic here to return the value based on the column index and the item type
        if (item instanceof Guerrier user) {
            return switch (columnIndex) {
                case 0 -> user.getNom();
                case 1 -> user.getRace();
                case 2 -> user.getClasse();
                case 3 -> user.getNiveau();
                case 4 -> user.getPdv();
                case 5 -> user.getPdf();
                default -> null;
            };
        } else if (item instanceof Soigneur user) {
            return switch (columnIndex) {
                case 0 -> user.getNom();
                case 1 -> user.getRace();
                case 2 -> user.getClasse();
                case 3 -> user.getNiveau();
                case 4 -> user.getPdv();
                case 5 -> user.getPdm();
                default -> null;
            };
        } else if (item instanceof Groupe user) {
```

Création de la Form

- Ensuite dans la classe java associée, il vous reste plus qu'à y ajouter dans le constructeur de la fenêtre, l'initialisation de votre table.

Définition des entêtes et classes des entêtes

```
private final String[] HEADER_GUERRIER = new String[] {"Nom", "Race", "Classe", "Niveau", "Pdv", "Pdf"}; 1 usage
private final String[] HEADER_SOIGNEURS = new String[] {"Nom", "Race", "Classe", "Niveau", "Pdv", "Pdm"}; 1 usage
private final Class<?>[] USER_COLUMN_CLASSES = {String.class, String.class, String.class, Integer.class, Integer.class, Integer.class}; 1 usage

private final String[] HEADER_GROUPE = new String[] {"Nom", "Nom du Guerrier", "Nom du Soigneur", "Date de création"}; 1 usage
private final Class<?>[] GROUPE_COLUMN_CLASSES = {String.class, String.class, String.class, String.class}; 1 usage
```

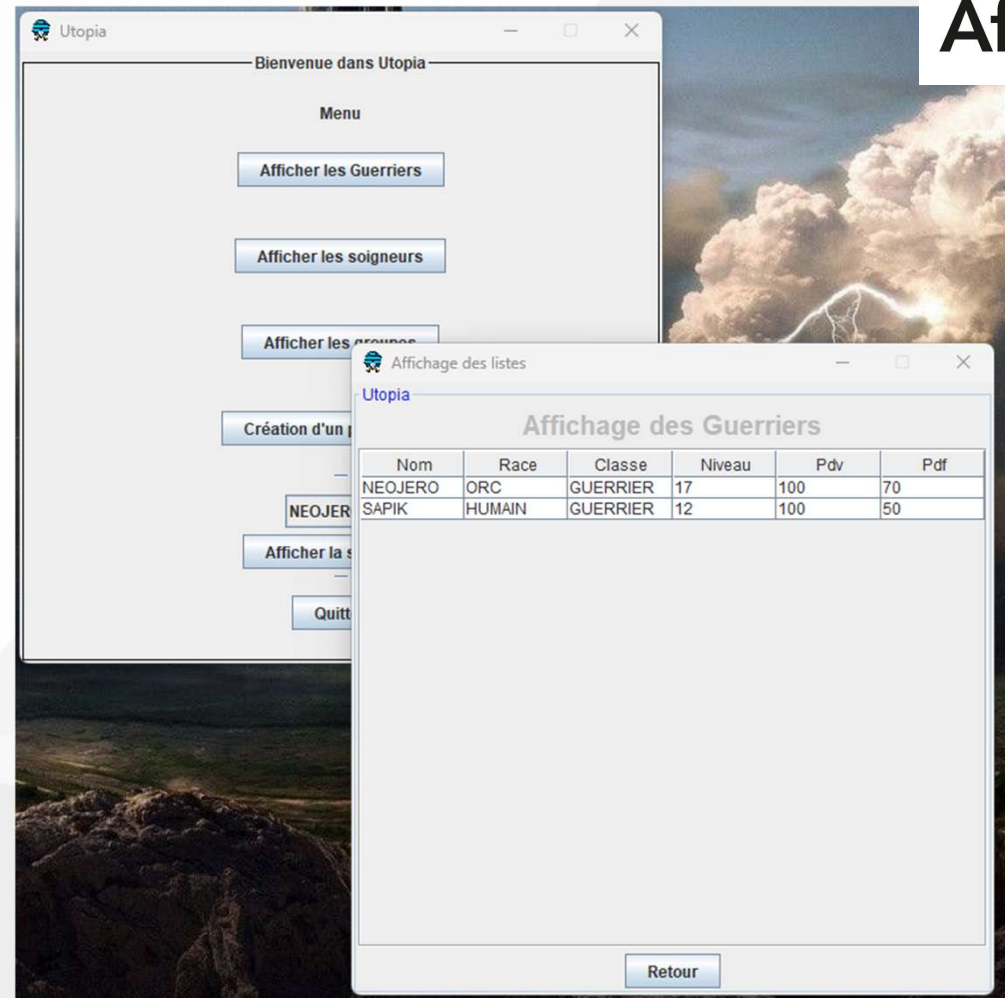
Appel de la méthode de construction de la table

```
affichage.setText("Affichage des Soigneurs");
configureTable(Soigneur.getSoigneurs(), HEADER_SOIGNEURS, USER_COLUMN_CLASSES);
```

Résultat

Vous obtenez un tableau récupérant les données de vos listes.

Concepteur Développeur d'Applications



A decorative graphic consisting of three overlapping shapes: a large green rounded rectangle on the left, a smaller magenta rounded rectangle to its right, and a green circle below the green rectangle. The text is positioned to the right of these shapes.

**Illustration d'une mise en
forme de Jtable
directement.**

Simplifions...

Vous pouvez simplifier la méthode sans passer par la redéfinition de la classe Modèle.

1. Je crée une méthode en fournissant une liste et un entête.

1

```
constructDataTable(Guerrier.getGuerriers(), HEADER_GUERRIER);
```

2. Ma méthode selon le type d'objets va construire la matrice en parcourant chaque élément de la liste.
3. Je fournis cette matrice et l'entête
4. Je fournis ma table à la scrollPane
5. Enfin, je revalidate et je repaint.

Concepteur Développeur d'Applications

2

```
private <T> void constructDataTable(List<T> dataList, String[] headers) { 1 usage

    // Création du tableau de données
    String[][] data = new String[dataList.size()][headers.length];

    // Remplissage du tableau selon le type d'objet
    for (int i = 0; i < dataList.size(); i++) {
        Object obj = dataList.get(i);
        if (obj instanceof Guerrier) {
            Guerrier g = (Guerrier) obj;
            data[i][0] = g.getNom();
            data[i][1] = g.getRace();
            data[i][2] = g.getClasse();
            data[i][3] = String.valueOf(g.getNiveau());
            data[i][4] = String.valueOf(g.getPdv());
            data[i][5] = String.valueOf(g.getPdf());
        } else if (obj instanceof Soigneur) {
            Soigneur s = (Soigneur) obj;
            data[i][0] = s.getNom();
            data[i][1] = s.getRace();
            data[i][2] = s.getClasse();
            data[i][3] = String.valueOf(s.getNiveau());
            data[i][4] = String.valueOf(s.getPdv());
            data[i][5] = String.valueOf(s.getPdm());
        } else if (obj instanceof Groupe) {
            Groupe gr = (Groupe) obj;
            data[i][0] = gr.getNom();
            data[i][1] = gr.getGuerrier().getNom();
            data[i][2] = gr.getSoigneur().getNom();
            data[i][3] = gr.getDateCreation().format(FORMATTER_DATE_FRENCH);
        }
    }

    // Initialisation de la table
    table = new JTable(data, headers);
    // ajout de la table dans le scrollpane
    scrollPaneTable.setViewportViewView(table);
    // on redefinie le panelTable avec les modifications
    panelTable.revalidate();
    panelTable.repaint();
}
```

3

4

5

Gestion des événements

Interaction avec nos fenêtres

Principes de gestion des événements

De manière générale, la gestion des événements en Java se fait de la manière suivante :

1. Les classes voulant recevoir une notification d'un événement enregistrent un écouteur auprès de la source d'événements.
2. L'écouteur est un objet dont la classe implémente une interface particulière pour traiter un événement comportant en général une seule méthode appelée pour notifier l'événement qui s'est produit.
3. La source d'événement (un composant, un périphérique d'entrée : souris ou clavier, ...) notifie les écouteurs enregistrés en appelant la méthode de l'interface particulière.
4. Cette méthode ne comporte en général qu'un seul argument dont la classe dérive de la classe `java.util.EventObject`.

- Pour un événement de type `type` (MouseWheel, Key, Mouse, ...) :
 - Une classe d'événement **`typeEvent`** dérivant de la classe **`java.util.EventObject`** contient toutes les informations sur l'événement notifié.
 - Un écouteur doit implémenter l'interface **`typeListener`** dont la ou les méthode(s) ont comme seul argument un objet de classe **`typeEvent`**.
 - La source d'événement possède deux méthodes publiques :
 - **`addtypeListener`** pour ajouter un écouteur de l'événement.
 - **`removetypeListener`** pour retirer un écouteur.
- En général, en interne, la source d'événement notifie les écouteurs enregistrés avec une méthode privée ou protégée nommée **`firetypeEvent`**.

Il existe un grand nombre de types d'évènements représentés par des classes dérivant de la classe `java.util.EventObject`.

Classe	Information(s) sur l'évènement définie(s)/ajoutée(s) par la classe
<code>java.util.EventObject</code>	Source de l'évènement : <code>public Object getSource();</code>
└ <code>java.awt.AWTEvent</code>	Identification de l'évènement (ID), consommation de l'évènement
└ <code>java.awt.event.ActionEvent</code>	Heure de l'évènement et modificateurs (boutons de souris, touches enfoncées : Shift, Alt, Ctrl...) (ID: ACTION_PERFORMED)
└ <code>java.awt.event.ComponentEvent</code>	Composant source de l'évènement (conversion de type)
└ <code>java.awt.event.InputEvent</code>	Heure de l'évènement et modificateurs (boutons de souris, touches enfoncées : Shift, Alt, Ctrl...)
└ <code>java.awt.event.KeyEvent</code>	Code de la touche, caractère généré (ID: KEY_TYPED, KEY_PRESSED, KEY_RELEASED)
└ <code>java.awt.event.MouseEvent</code>	Position de la souris, nombre de clics, indicateur de menu popup (ID: MOUSE_CLICKED, MOUSE_PRESSED, MOUSE_RELEASED, MOUSE_MOVED, MOUSE_ENTERED, MOUSE_EXITED, MOUSE_DRAGGED, MOUSE_WHEEL)
└ <code>java.awt.event.MouseWheelEvent</code>	Rotation de la molette de la souris (nombre, pixels de défilement)

Ajout d'un évènement

On va commencer par la méthode à base d'**ActionListener**.

Comment faire ça ? En ajoutant à notre bouton **addActionListener** et en redéfinissant la méthode **actionPerformed**

L'écriture proposée est une classe anonyme. Il s'agit d'une écriture simplifiée.

En fait, c'est très simple : la méthode **addActionListener** a besoin d'un objet de type **ActionListener**, on en dérive donc une classe anonyme qui redéfinit quelques méthodes (ici une) est qui directement utilisée pour créer un unique objet d'écoute.

Illustration

- Ajout de deux évènements sur nos boutons valider et annuler
- Bonne pratique : afin de ne pas mélanger le code lié à Swing et la logique de code, on fera en sorte de créer une méthode concernant l'action à réaliser.
- Ensuite dans les méthodes **valider()** et **quitter()**, il suffit d'implémenter le code permettant de valider l'action.

```
// ajout d'évènement sur les boutons
valid.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {

        valider();

    }
});
JButton cancel = new JButton("Annuler");
cancel.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {

        quitter();

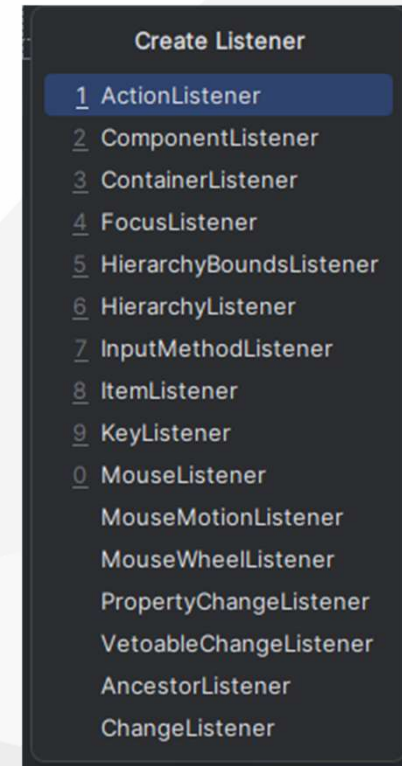
    }
});
```

Depuis l'IDE

Depuis notre IDE, nous pouvons voir tous les événements disponibles.

Depuis un composant de notre UI, en effectuant un **clic-droit -> create listener**, on peut ajouter automatiquement le code dans notre classe java.

Ainsi, il nous plus qu'à définir les méthodes contenant l'implémentation de l'événement.



Gestion d'apparence

donner un peu de style à nos UI

Illustration : méthode de gestion du style

```

public static void main(String[] args) {
    // Gestion du style des fenêtres
    try {
        initLookAndFeel(LOOKANDFEEL);
    } catch (ClassNotFoundException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (InstantiationException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (IllegalAccessException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    } catch (UnsupportedLookAndFeelException e) {
        // TODO Auto-generated catch block
        e.printStackTrace();
    }

    EventQueue.invokeLater(new Runnable() {
        public void run() {
            try {
                FirstApp window = new FirstApp();
                window.frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
}

```

```

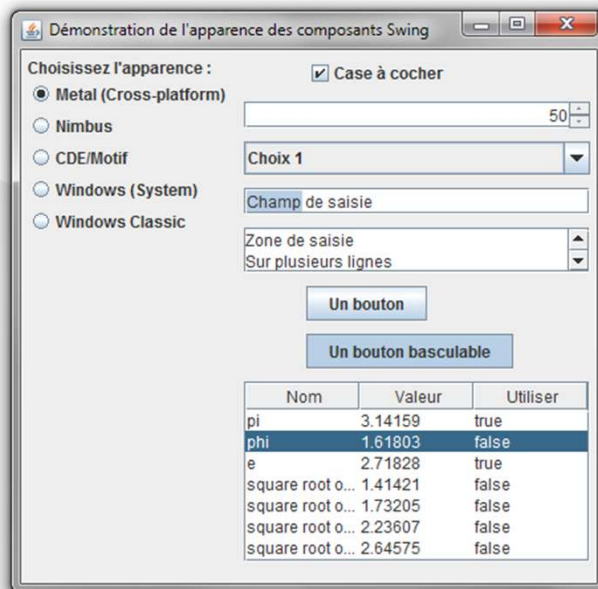
/**
 * Methode de gestion des styles
 * @throws UnsupportedOperationException
 * @throws IllegalAccessException
 * @throws InstantiationException
 * @throws ClassNotFoundException
 */
private static void initLookAndFeel(String name) throws ClassNotFoundException,
    InstantiationException,
    IllegalAccessException,
    UnsupportedLookAndFeelException {

    for (UIManager.LookAndFeelInfo laf : UIManager.getInstalledLookAndFeels()) {
        if (name.equalsIgnoreCase(laf.getName())) {
            UIManager.setLookAndFeel(laf.getClassName());
            laf_selected = name;
        }
    }
}

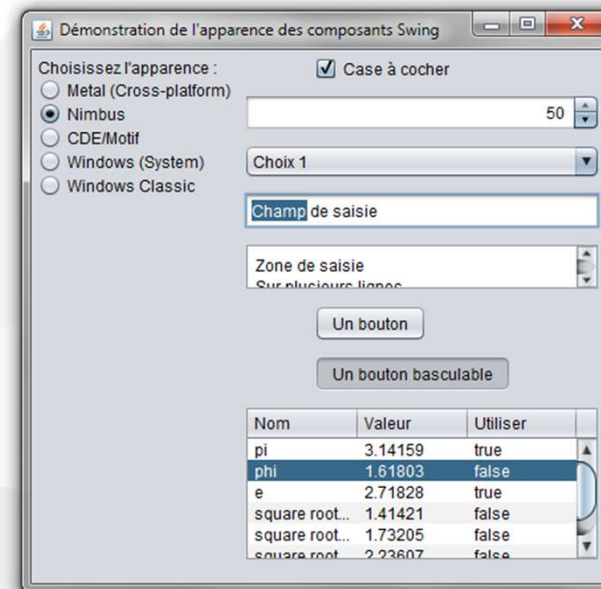
```

Lookandfeel

Metal

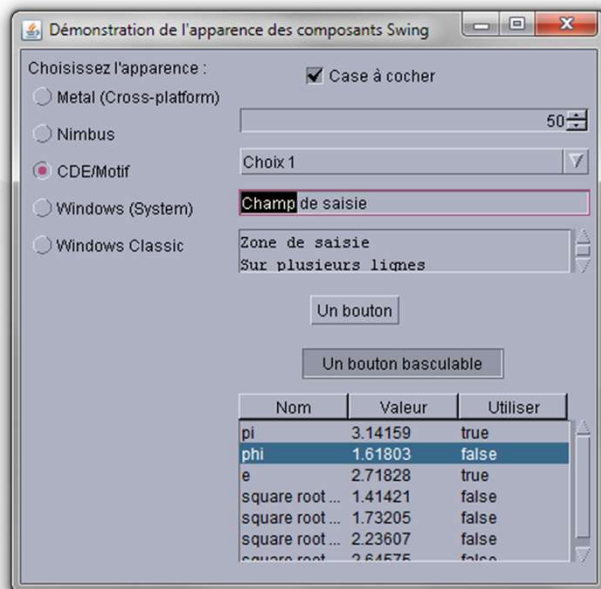


Nimbus

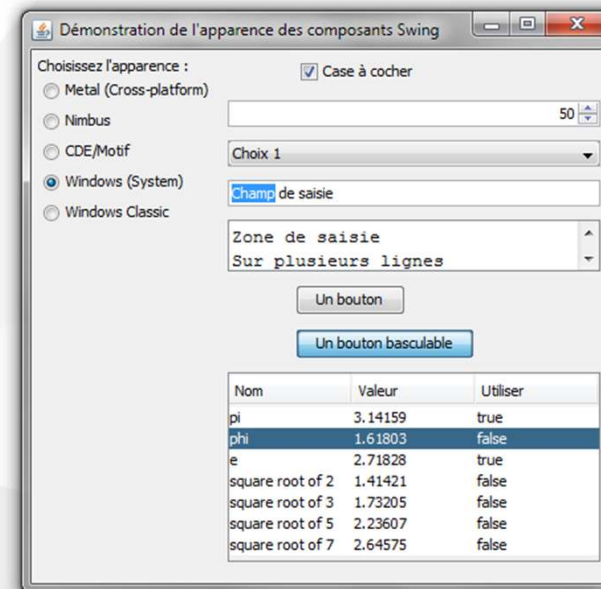


Lookandfeel

CDE/Motif

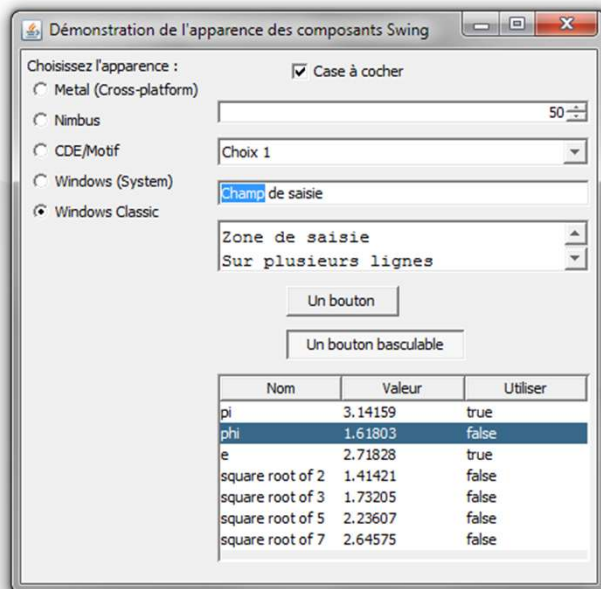


Windows



Lookandfeel

Windows Classic





Créer une UI depuis notre IDE IntelliJ avec Swing et UI DESIGNER

<https://www.jetbrains.com/help/idea/design-gui-using-swing.html>

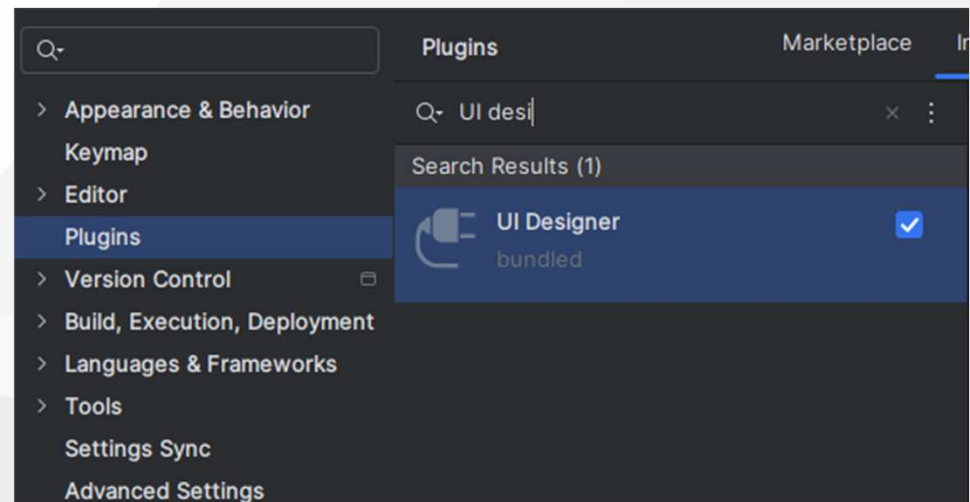
En utilisant `UI Designer`, vous pouvez rapidement créer des dialogues et des groupes de contrôles à utiliser dans des conteneurs de haut niveau, tels que `JFrame`.

UI DESIGNER

UI Designer est fourni et activé dans IntelliJ IDEA par défaut.

- Si les fonctionnalités ne sont pas disponibles, assurez-vous que vous n'avez pas désactivé le plugin.
1. Appuyez Ctrl + Alt + S puis sélectionnez Plugins.
 2. Ouvrez l'onglet Installé, trouvez le plugin UI Designer et sélectionnez la case à cocher à côté du nom du plugin.

Verifier que le plugin est présent.



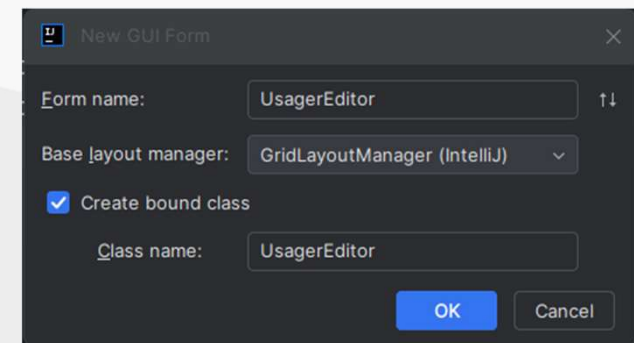
Création d'une nouvelle UI

.form

- IntelliJ stocke les informations sur votre UI dans un fichier .form
 - C'est ce fichier qui est lu par le plugin et qui va vous permettre de designer votre interface.
- Il est associé à une classe Java du même nom et c'est dans cette classe que vous allez définir l'implémentation des différentes interactions avec l'application.

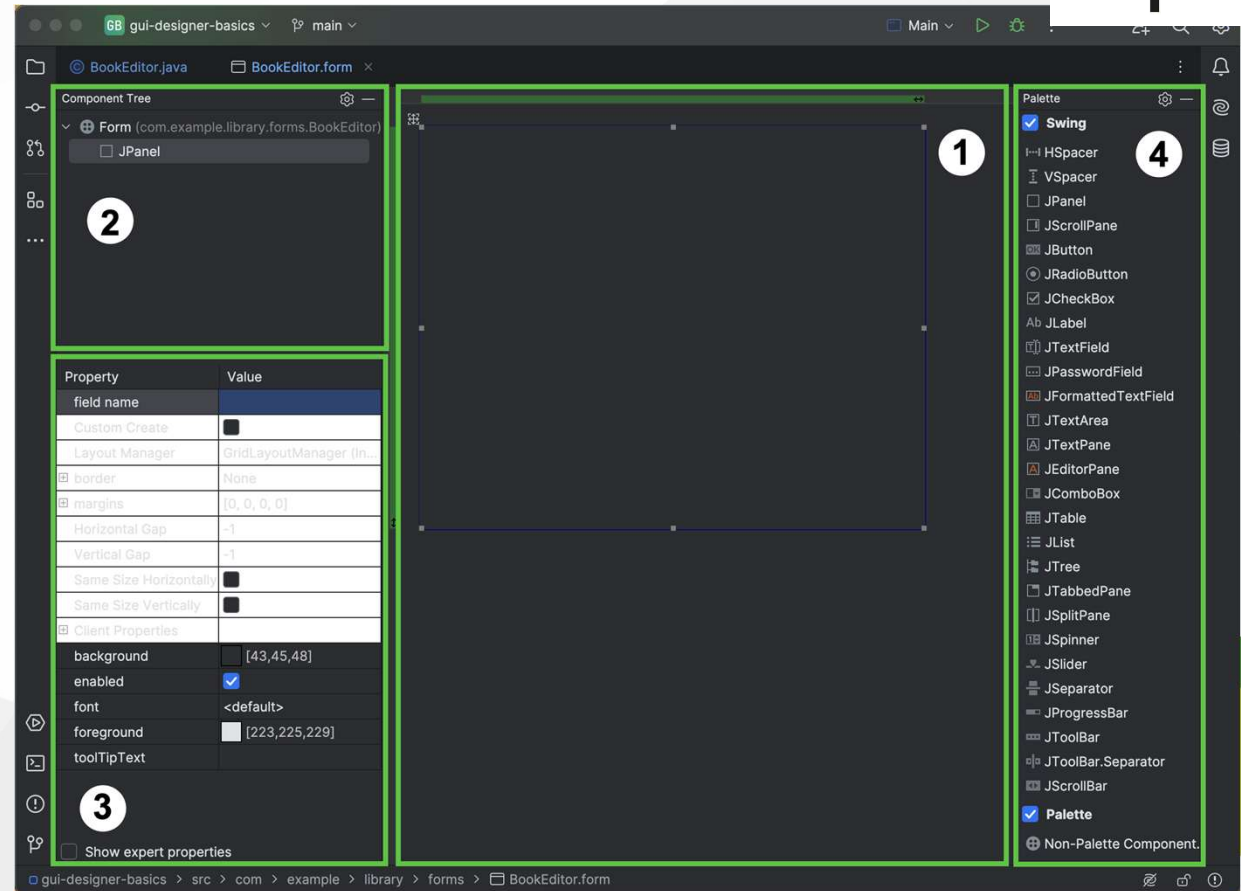
Création d'une nouvelle interface

- Pour créer une nouvelle interface depuis IntelliJ
- Depuis le package source (vue),
 - clic droit -> new... -> Swing UI Designer
- Dans la nouvelle fenêtre de dialogue, indiquer
 - Form name : le nom de votre UI
 - Base Layout Manager : GridLayoutManager (IntelliJ)
 - Create bound class : sélectionner l'option avec de lier le .form avec la classe .java associée



UI DESIGNER

1. **Espace de travail de formulaire** : montre une représentation visuelle des composants de votre forme. Le composant JPanel de premier niveau est ajouté par défaut lorsque vous créez un nouveau formulaire GUI dans IntelliJ IDEA, et vous pouvez ajouter plus de composants pour construire votre formulaire.
2. **Arbre à composants**: indique les composants contenus dans le formulaire de conception et vous permet de naviguer et de sélectionner un ou plusieurs composants. Notre formulaire ne comporte actuellement qu'un seul élément, JPanel, mais nous allons bientôt ajouter d'autres composants.
3. **Les propriétés des composants**: affiche les propriétés du composant actuellement sélectionné dans l'espace de travail de la forme ou l'arbre à éléments.
4. **Palette** : contient une liste des composants UI que vous pouvez placer sous une forme.



-
- The screenshot shows a Java Swing IDE with the following components:
- Component Tree:** A tree view on the left showing the hierarchy of components. The root is `Form (com.example.library.forms.BookEditor)`, and it contains a single child component, `JPanel`.
 - Properties Window:** A panel on the left showing the properties of the selected `JPanel` component. The `field name` property is highlighted with a red box and contains the text `contentPane`. Other visible properties include `Custom Create` (unchecked), `Layout Manager` (set to `GridLayoutManager (Inherits from JPanel)`), `border` (set to `None`), `margins` (set to `[0, 0, 0, 0]`), `Horizontal Gap` (set to `-1`), `Vertical Gap` (set to `-1`), `Same Size Horizontally` (unchecked), `Same Size Vertically` (unchecked), `Client Properties` (expanded), `background` (set to `[43,45,48]`), `enabled` (checked), `font` (set to `<default>`), `foreground` (set to `[223,225,229]`), and `toolTipText` (empty).
 - Palette:** A panel on the right showing a list of Java Swing components available for addition to the design. The `Swing` category is selected, and the list includes `HSpacer`, `VSpacer`, `JPanel`, `JScrollPane`, `JButton`, `JRadioButton`, `JCheckBox`, `JLabel`, `TextField`, `JPasswordField`, `JFormattedTextField`, `JTextArea`, `JTextPane`, `JEditorPane`, `JComboBox`, `JTable`, `JList`, `JTree`, `JTabbedPane`, `JSplitPane`, `JSpinner`, `JSlider`, `JSeparator`, `JProgressBar`, `JToolBar`, and `JToolBar.Separator`.
 - Design View:** A large central area showing a single, empty `JPanel` component, which is the visual representation of the component tree.

Ajout de composants

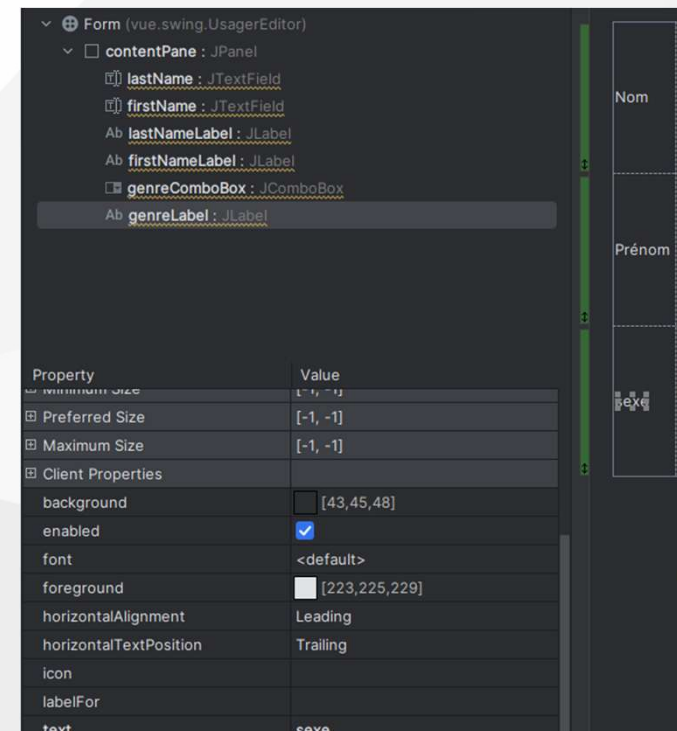
Depuis le .form, vous pouvez dès lors ajouter les composants de votre UI

Par exemple :

- Un champs texte pour le nom de l'abonné
- Une champs texte pour le prenom
- Une comboBox pour le genre

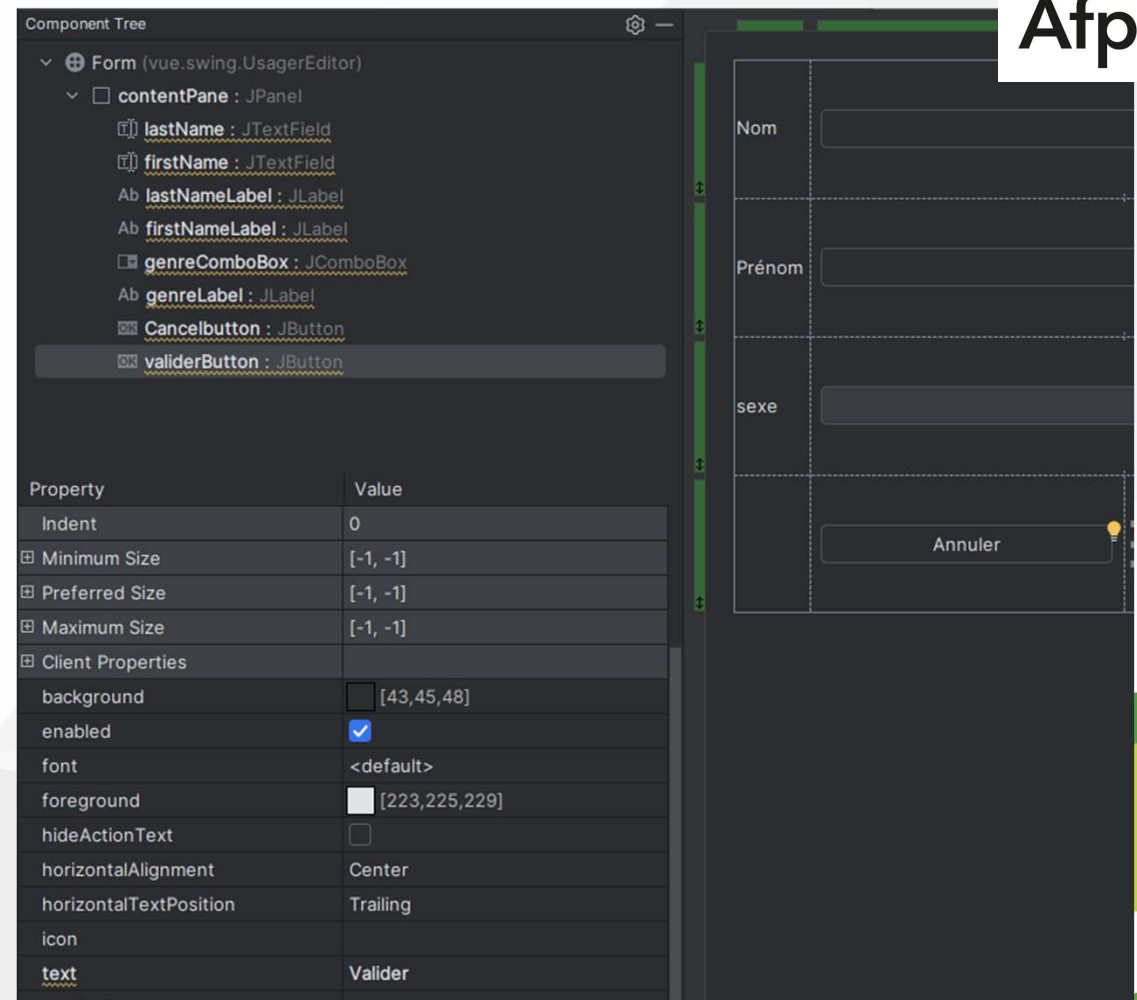
Note : Le fait d'ajouter les éléments dans le .form sont automatiquement répercuté dans le .java

- Ensuite, il faut aller renommer les éléments correctement car par défaut, il nomme les éléments soit le nom du composant incrémenté s'il y'en a plusieurs.
 - Soit directement depuis les propriétés des composants
 - Soit depuis le .java



Ajout de boutons

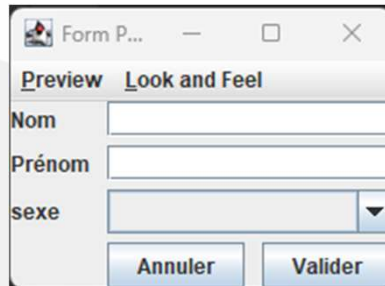
1. Ajouter le composant VSpacer sous la cellule avec la JCheckBox. Cela créera une zone pour les boutons et le divisera également en deux moitiés.
2. Ajouter deux éléments JButton de chaque côté de l'entretroise verticale.
3. Dans la propriété du texte de chaque bouton, spécifiez leur nom en conséquence.



Prévisualiser votre UI

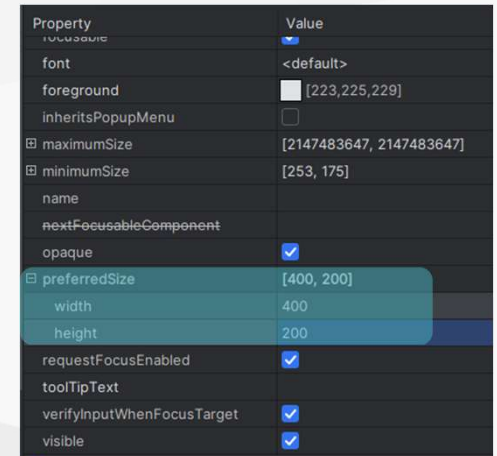
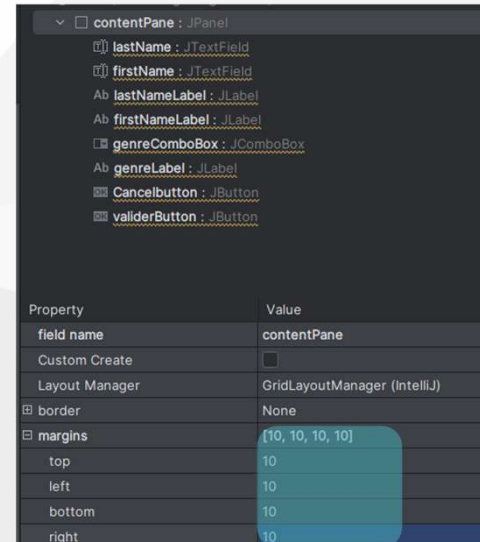
Preview

- Cliquez avec le bouton droit sur votre formulaire GUI et sélectionnez Preview dans le menu contextuel.



Améliorer l'aspect de votre UI

- Sélectionner le JPanel contenant l'ensemble de vos composants
- Vous pouvez ajouter des marges et modifier la taille



Mise en place de notre interface dans l'application

Après avoir designer notre UI, nous pouvons l'utiliser dans l'application.

Pour cela, il faut construire l'objet et la rendre visible.

- Un frame est un objet comme tous les objets avec ses propriétés dont le titre, sa visibilité etc..
- Elle doit tout d'abord hérité de la classe `JFrame`
- Il faut donc écrire le constructeur de notre frame et y implémenter les différents évènements liés aux composants etc...

Concepteur Développeur d'Applications

```
public UsagerEditor() { no usages new *

    setTitle("Usager Editor");
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setContentPane(contentPane);
    pack();

    // Set the frame location to the center of the screen
    setLocationRelativeTo(null);

    // Save button event listener
    validerButton.addActionListener(new ActionListener() { new *
        @Override new *
        public void actionPerformed(ActionEvent e) {
            saveChanges();
        }
    });

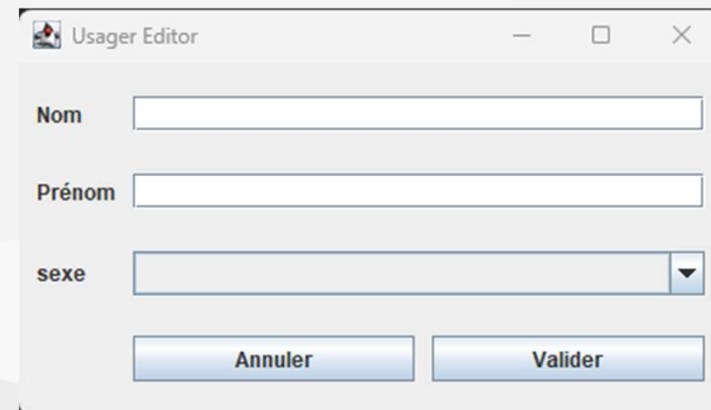
    // Cancel button event listener
    Cancelbutton.addActionListener(new ActionListener() { new *
        @Override new *
        public void actionPerformed(ActionEvent e) {
            cancelChanges();
        }
    });
}
```

Exécution

Dans ma classe Main

```
private void start() { 1 usage  neojero *  
    InputAndDisplay.message( message: "démarrage", type: 1);  
    //MenuFrame.acceuil();  
  
    //BookEditor bookEditor = new BookEditor();  
    //bookEditor.setVisible(true);  
  
    UsagerEditor usagerEditor = new UsagerEditor();  
    usagerEditor.setVisible(true);  
}
```

Ma fenêtre UI



The screenshot shows a Java Swing window titled "Usager Editor". It contains three text input fields labeled "Nom", "Prénom", and "sexe". The "sexe" field is a dropdown menu. At the bottom, there are two buttons: "Annuler" (Cancel) and "Valider" (Validate).

<https://waytolearnx.com/2020/05/creation-interface-graphique-avec-swing-les-bases.html>



Merci

afpa.fr



Jérôme BOEBION

Version 2.0 - 2024